



UNIVERSITY OF BOLOGNA
MASTER'S DEGREE IN COMPUTER SCIENCE AND
ENGINEERING

Lab-activity-04

Laboratory Report

Student:

Sajmir Buzi

Academic Year 2025/2026

0.1 Objective

The objective of this activity is to implement a behaviour-based controller in AR-GoS using the motor schemas architecture. The robot has to reach a light source while avoiding collisions with walls, boxes and other robots.

The controller is written in Lua and uses the foot-bot light sensors, proximity sensors, LEDs and differential steering wheels. The final behaviour is obtained by combining simple motor schemas represented as vectors in polar coordinates.

0.2 Motor Schemas

The controller is based on three main schemas: cruise, light attraction and obstacle repulsion. Each schema returns a vector with two fields: `length` and `angle`. The vectors are then summed using the function `vector.vec2_polar_sum`.

The cruise schema gives the robot a small forward force, with length `0.5` and angle `0.0`. This prevents the robot from stopping when the light stimulus is weak.

The light attraction schema reads all light sensors. For every active sensor, it creates a vector with the same direction as the sensor and with a length proportional to the perceived light value. All these vectors are summed to obtain the final attraction vector.

The obstacle avoidance schema reads the proximity sensors and creates repulsive vectors in the opposite direction of the detected obstacles. The proximity value is squared and multiplied by `2.0`, so close obstacles have a stronger effect than distant ones.

0.3 Target Behaviour

When the robot gets very close to the light, simply following the light vector may cause oscillations. This happens because the perceived direction of the light changes quickly when the robot is near the source.

To avoid this problem, the function `near_light()` checks the maximum value perceived by the light sensors. If this value is greater than `NEAR_LIGHT_VALUE`, the robot is considered to be near the target. When this condition is true and no obstacle is very close, the robot stops translating and rotates slowly on itself by setting opposite wheel velocities. This keeps the robot close to the light source without moving far away from the target area.

0.4 Wheel Velocity Computation

In each simulation step, the controller computes the cruise vector, the light attraction vector and the obstacle repulsion vector. The three vectors are summed to obtain the final desired motion direction.

The resulting vector is converted into a translational velocity v and an angular velocity ω . Then these values are converted into left and right wheel velocities using the differential steering equations.

The laboratory specification states that the wheel velocity cannot exceed 15. For this reason, after computing the two wheel velocities, the controller scales them if one of them is above the maximum allowed value. This keeps the direction of the action while respecting the physical constraint.

0.5 LED Feedback

The LEDs are used only as a visual indicator and do not affect the movement of the robot. The controller checks the proximity sensors and turns the LEDs red if an obstacle is very close. Otherwise, the LEDs remain green.

0.6 Conclusion

The implemented controller follows the motor schemas architecture required by the assignment. The robot combines a forward cruise behaviour, an attraction behaviour towards the light and a repulsion behaviour from obstacles.

The resulting controller is simple and reactive: it does not build a map of the environment, but it decides the wheel velocities directly from the current sensor readings. The robot can move towards the light source, avoid obstacles and remain close to the target by rotating slowly near it.

The wheel velocity limit is explicitly enforced in the code, so the controller respects the physical constraint required by the laboratory specification.